

Extra Capstone: City Disruption Memory

An AI-operated data engineering platform using live city data

Question: What reported infrastructure issues and transit disruptions overlapped around a stop or corridor, what changed during the week, and what evidence did the system have at the time?

User: a local reporter, community researcher, or resident investigating a specific area. The product offers an evidence timeline, not safety advice or guaranteed travel directions.

Working city: Chicago, because official 311, transit-alert, and transit-geography sources are available. The project can later support another city through adapters. Build one city first.

Deliverable in this folder: a build specification and learning schedule. The application, collectors, and dataset are not implemented by this document.

Primary learning objective: configure, deploy, diagnose, and operate a data platform with AI assistance. Spark, Docker, Terraform, Prometheus, Grafana, and an evidence-grounded AIOps workflow are core requirements. City-data extraction and explanations are secondary features.

Engineering question: when ingestion, a Spark job, or infrastructure fails, can an AI assistant identify the likely cause from operational evidence, propose a bounded fix, and verify recovery without losing or duplicating data?

Commitment	Schedule
Core AI course	Keep D01-D60, 12 weeks, 360 hours, and the existing support capstone
Extra city-data capstone	Proposed D61-D75, three additional five-day weeks
Daily study time	Six hours, with weekends off
Extra study hours	90
Combined course + extension	75 study days, 15 weeks, 450 hours

The extension is additional work, not a hidden task squeezed into the original 60 days. Its 90-hour schedule is a target, not a guarantee, and assumes you can reuse Airflow, FastAPI, evaluation, logging, and structured-output components. The infrastructure/AIOps work replaces the earlier emphasis on an elaborate city UI and a large text-extraction evaluation; it does not replace the core 60-day course. If platform setup, source access, or a

completion gate takes longer, extend the finish date instead of omitting a required component or substituting synthetic city findings.

1. What makes this your project

You will not download a finished city-analysis dataset or clone a tutorial pipeline. You will collect official operational records, retain source changes, construct the stop/corridor overlap dataset, label real examples, and publish your methodology.

The proposed contribution is the **connection between collected city-data history, pipeline execution, configuration changes, and evidence-backed incident recovery**. Retain the derived disruption history as the real workload. The city still owns its source records; collection does not transfer their copyright or remove their terms.

Defensible differentiation

Existing projects already analyze Chicago bus reliability:

- [CTA StopWatch](#) combines collected bus locations, schedules, and geography to measure service reliability.
- [Bus Pending](#) collects bus positions and analyzes delay patterns with schedules and demographic information.

Do not pitch another bus-delay dashboard as new. This capstone combines cross-source disruption evidence with an operational incident trail: which source versions a job processed, which infrastructure/configuration version ran it, which alerts appeared, what action was approved, and whether recovery preserved data correctness. Temporal modelling and AIOps have established precedents; this combination and your measured experiments define the contribution, not a claim of global uniqueness.

Your portfolio claim after completing the project can be: “I built a Docker/Terraform-managed Spark data platform for live Chicago records, instrumented it with Prometheus and Grafana, and evaluated an AI incident assistant against controlled failures with approval-gated recovery.” Claim cloud deployment, successful recovery, and performance improvements only when your saved runs establish them.

2. Source contracts and real-data boundaries

Source	Official entry point	Use	Important limit
Chicago 311 service requests	Dataset · API sample · Metadata	Report type, location, administrative status, source timestamps	A complaint is a report, not a verified physical obstruction. Status Completed does not prove the street is clear.
CTA Customer Alerts	Official API documentation	Planned/unplanned service events, affected services, descriptive text	Preserve structured fields first; AI extracts only facts the text supports. Capture a full successful feed before inferring absence.
CTA GTFS	Official GTFS documentation · Feed download	Stops, route IDs, stop-to-route membership, schedule/geography versions	Scheduled service is not actual service. Retain feed versions so current geography does not rewrite history.

Source verification performed while planning: a live two-row 311 response contained sr_number, sr_type, status, created_date, last_modified_date, and closed_date; only one sampled row contained coordinates. The official metadata warns that 311 INFORMATION ONLY CALL locations often identify the 311 center rather than the caller's issue. Exclude that category from spatial analysis. CTA documentation identifies an alerts API and GTFS distribution. CTA payload/GTFS ingestion has not been executed here.

Terms: Chicago's metadata uses SEE_TERMS_OF_USE, not a blanket open-source data license. CTA links a [Developer License Agreement and Terms of Use](#). Review these before collection and redistribution. Do not claim your code license covers source data or assume every snapshot can be published. Document attribution, retention, and permitted sharing; publish derived aggregates or acquisition scripts where raw redistribution is restricted.

Bound the first build

Choose one corridor crossing no more than three adjoining community areas and at most 30 stops after inspecting GTFS. Start with three directly relevant 311 types, such as street-light, traffic-signal, and pothole reports; verify their current exact type values. Store other categories outside the analytical model only if needed for source completeness.

Do not add crime data, individual rider tracking, route optimization, real-time vehicle tracking, weather, or construction permits to this capstone. Those create separate questions and source contracts. Add a source later only if it changes a defined user decision.

Collect your own observations

Start collectors on D62 and retain at least seven calendar days of actual observations by the final demo. Unattended jobs may run on weekends; you do not need weekend study sessions. Keep collectors active after the build to develop a longer history.

A starting cadence, subject to each source's current limits, is CTA alerts every 15 minutes, scoped 311 changes every two hours, and a daily GTFS version check. Log failed polls and rate-limit responses. Reduce cadence if required; freshness metrics must use the configured cadence.

Backfill up to 30 days of relevant 311 records for context. **Backfilled current records do not establish what the API showed in the past.** Your historical “known at” feature starts with your first successful collection. Do not manufacture historical CTA versions or describe today's version as yesterday's knowledge.

If the collection window contains no alert corrections or genuine cross-source overlaps, show the honest empty result and keep collecting. Small synthetic fixtures are allowed for tests, but not as the product dataset or as proof of a real city incident. Record a real correction/overlap only when one is observed; its occurrence is not guaranteed by the schedule.

3. Product behavior

Provide two small surfaces rather than a large city dashboard:

1. **City evidence view:** stop/corridor filters, current versus historical records, source changes, provenance, and freshness. A table and a short timeline are sufficient.
2. **Operations view:** Grafana dashboards plus an incident page showing the affected DAG/job, logs, metrics, source/configuration versions, diagnosis, proposed patch/action, approval, and recovery evidence.

Keep city queries deterministic. Natural-language city chat and a polished map are not completion requirements. An optional cited city explanation must use selected evidence and must not issue safety guarantees.

Separate the two timelines

- **Source-effective time:** when the provider says an alert or status applies. Missing ends remain unknown, not an invented resolution.
- **Observation/knowledge time:** when your system captured a source version. For AI-derived facts, knowledge time is the later of capture time and the first persisted validated extraction. Preserve human-review time as well.

For a historical view, use only raw versions, geography, and derived facts available by the requested cutoff. A model run performed today cannot appear in a reconstructed answer supposedly generated yesterday. Offer a separate, labelled retrospective reanalysis if you process old evidence with a new model.

The 311 created_date/closed_date interval can indicate the lifetime of the **administrative report**. It must not be labelled the physical hazard's duration. last_modified_date helps ingestion; it does not reconstruct every prior status transition.

4. Data platform and infrastructure

Component	Required role	AI assistance to demonstrate
Python collectors + Airflow	Acquire permitted live inputs; schedule Spark jobs, checks, and bounded replay	Diagnose failed tasks using run state, source coverage, and logs
Apache Spark	Normalize raw records into Parquet, deduplicate record versions, and reprocess captured history	Propose and benchmark executor memory, partition counts, and skew mitigation
Docker + Compose	Run a Spark master and two worker containers plus the supporting services on one machine	Prepare reviewed image/resource/network configuration patches and diagnose startup failures
Terraform	Provision a named local Docker network and persistent volumes through the Docker provider	Draft infrastructure patches, validate them, and explain the saved plan before approval
PostgreSQL + PostGIS + dbt	Store source history, geography, deterministic overlap marts, and data checks	Explain a failed contract or query from its actual evidence
Prometheus + Grafana	Collect platform/job metrics, display dashboards, and raise actionable alerts	Correlate alert timelines with failed runs and configuration changes
FastAPI + a small UI	Show evidence and expose the incident investigation workflow	Present cited diagnosis, uncertainty, proposed actions, and verification results
Ollama / an open-weight model	Perform read-only operational diagnosis and prepare bounded proposals	Compare with deterministic runbook diagnosis; record model/license/resource needs

Deployment boundary: start on a single local host. Two Docker workers let you exercise Spark scheduling and failure behavior; they do not demonstrate multi-host availability or production scale. Select pinned versions and resource budgets on D61 after inspecting available hardware. Run the local model on demand if simultaneous model/Spark workloads exceed that budget; record the effect on diagnosis latency.

Terraform ownership: Terraform owns the named network and volumes; Compose references them as external resources and owns service containers. Do not let both tools manage the same object. Keep state and credentials out of version control; use stable resource names and persist the reviewed plan/configuration hash in the incident trail. Learn resource graphs, state, drift, lifecycle, validation, and plan review through actual local resources, not a fake cloud deployment.

Cloud boundary: a later cloud deployment requires a provider, account permissions, region, cost ceiling, storage/IAM design, and approval. It is outside this local 90-hour target. Do not claim AWS/Azure/GCP provisioning from a Docker-provider exercise. Require approval before infrastructure apply/destroy or permission changes, including local changes that risk persistent data.

Storage and computation: preserve compressed JSON/XML snapshots and original GTFS archives on persistent volumes. Spark writes normalized, versioned Parquet outputs. Publish validated records to PostgreSQL through idempotent staging/merge; dbt/PostGIS computes spatial/time overlaps. PostGIS remains authoritative for geographic distance; Spark does not need a new spatial framework.

Flow: live collectors → immutable evidence → Spark normalization/replay → validated history → SQL/PostGIS marts → city evidence view. In parallel, metrics/logs/run records → Prometheus/Grafana alert → AI diagnosis → reviewed patch/action → approval → bounded execution → recovery checks → incident history.

The bounded city dataset does not require Spark for throughput. Spark is an explicit operations-learning objective here. Benchmark captured-data replay with documented multiplicity and separate namespaces; never present duplicated replay records as new observations or let them enter the city findings. Do not add Kafka, Kubernetes, a vector store, or several agent frameworks without a demonstrated need.

Stored entity	Required fields / behavior
collection_runs	Source, scope, start/end, outcome, pages, count, checkpoint, completeness, failure reason
raw_snapshots	Snapshot ID, sanitized source query, capture UTC, response metadata, SHA-256, path, collection-run ID
source_versions	Source/record ID, canonical hash, snapshot reference, source timestamps, observed interval; append corrections
geography_versions	GTFS hash, capture metadata, stop/route geometry and membership, available service dates
pipeline_runs	Airflow/Spark run IDs, input manifests, code/image/config hashes, Terraform plan/apply reference, outputs, start/end, outcome
incidents	Alert onset, affected runs, metric/log references, known configuration, injected-fault label where applicable, diagnosis and uncertainty
remediation_actions	Proposed patch/action, evidence, model/prompt version, approver/time, allowed scope, execution result, rollback and verification
disruption_overlaps	Separate source-version IDs, stop/corridor, geography version, matching rules, distance, certainty, derivation time

Stored entity	Required fields / behavior
extracted_facts / explanations	Optional city-AI outputs with input evidence IDs, supported spans/citations, model/prompt version, validation and availability time

Incremental collection and replay

For 311, page by a stable order using modification time plus record ID, with an overlapping lookback for equal timestamps and delayed updates. Advance checkpoints only after raw evidence is durable and the corresponding processing outcome is recorded. Deduplicate by source/record/version. Inspect the actual API contract before choosing pagination syntax.

For alerts, retain complete-feed captures and record-level versions. Absence from a partial or failed poll is unknown. Absence from a successful complete feed means “no longer observed,” not physical resolution. Use bounded reconciliation to expose missed updates and deletions.

Publish Spark outputs atomically through a completed manifest and idempotent database staging/merge. Retry a failed run against pinned inputs and configuration; verify both record-level correctness and checkpoint state. Duplicate polls retain freshness evidence without adding business versions. Persist any AI output used in authoritative history.

Spatial and temporal matching

Use provider IDs and the GTFS version available at the knowledge cutoff. Keep route-wide alerts distinct from precise locations; quarantine unsupported/ambiguous locations. Use ST_DWithin on geography values in meters, initially 300 m, and compare 150/300/500 m sensitivity. Proximity is not walking distance or proof of impact.

Calculate temporal overlap in SQL from documented intervals. Administrative report closure does not establish physical resolution; nearby records do not establish causation. Preserve separate IDs and uncertainty.

Normalize timestamps to UTC and display America/Chicago. Preserve source strings and documented timezones; quarantine ambiguous daylight-saving timestamps rather than choosing an offset.

5. AIOps and AI-assisted configuration

Diagnostic evidence and monitoring

Instrument the chosen Spark runtime through its supported metrics endpoints or a configured exporter, then verify actual Prometheus scrapes on D65. Capture job/stage duration, executor/worker health, failed tasks, memory/GC pressure where exposed, container resource use, Airflow outcomes, input freshness, quarantine counts, and publication/checkpoint status. Unsupported metrics remain explicit gaps, not invented values.

Build Grafana dashboards for platform health, Spark execution, and data health. Each actionable alert needs a threshold tied to cadence/resource limits, a duration, an affected service/job, and a runbook link. Correlate runs through logged IDs and stored manifests; avoid record IDs or unbounded run IDs as Prometheus label values.

The assistant receives a sanitized evidence bundle: alert, bounded log excerpts, relevant metric window, DAG/Spark run state, input coverage, current configuration, and recent approved changes. It returns likely cause, cited evidence, uncertainty, missing checks, and a proposed action. Keep observations separate from hypotheses; compare timestamps instead of assuming the newest change caused the failure.

Configuration assistance and action boundaries

- **Spark:** propose memory/partition settings or skew mitigation with an explicit hypothesis. Benchmark unchanged inputs against the baseline; retain unsuccessful proposals.
- **Docker:** propose image, resource, or network changes as patches. Validate Compose configuration, inspect the diff, and apply only after approval.
- **Terraform:** generate a patch and run validation/plan under controlled tooling. A plan is not authorization to apply; review replacements, state effects, and persistent-data risk.
- **Recovery:** propose retry, bounded backfill, or a known configuration rollback with exact run/input scope. Verify idempotency and downstream publication before calling the incident resolved.

Start read-only. The model must not receive a Docker socket, infrastructure credentials, unrestricted shell, or direct permission to apply/destroy infrastructure. Source text and logs are untrusted data, not tool instructions. Redact credentials and unnecessary addresses before model access.

Use deterministic wrappers for approved actions: typed arguments, allowed command/action IDs, resource/run allowlists, timeouts, and audit records. Human approval names the exact target and change immediately before execution. Only a separately preauthorized, idempotent retry within fixed limits may run without a new approval; uncertain or destructive changes stop for review. An assistant failure leaves dashboards, runbooks, and manual operations usable.

Prove operational value

Run four controlled incident classes in an isolated namespace: Spark executor memory pressure, partition skew, an upstream-schema change in a captured-input copy, and interrupted collection/publication. Do not modify public sources, corrupt the live raw archive, fill the host disk, or interrupt unrelated services. Use recorded city data for the workload; mark all injected faults and load replay as experiments.

Create development scenarios and freeze a different variant of each class for held-out evaluation before prompt tuning. Record the intended fault separately from the assistant's input. Capture the actual failure, symptoms, injected/configuration version, diagnosis, approved action, and outcome. If the runtime behaves differently from the intended fault, report what happened rather than assigning the expected diagnosis.

Compare a deterministic alert/runbook baseline with the AI assistant using the same evidence. Measure supported diagnosis, unsupported claims, abstention, unsafe-action proposals, time to diagnosis, time to verified recovery, duplicate/missing business versions, and resource overhead. Repeat comparable runs where practical and record sample size, timing boundaries, machine limits, and failures; do not infer production reliability from a few local experiments.

Recovery requires a healthy rerun, cleared/recovered telemetry under the alert policy, fresh expected inputs, correct output versions, and a valid checkpoint. A quieter dashboard alone does not prove recovery.

Secondary city-data AI

Alert-prose extraction and cited city explanations are optional after the operational gates pass. Prefer structured provider fields, require evidence spans/citations, preserve extraction availability time, and leave uncertain fields null. Do not spend the infrastructure/AIOps allocation on a 40-case extraction study or a generic chatbot. The operational evaluation above is the required AI evaluation.

6. Three-week build schedule

Each row uses **1.5 h learn/design + 3 h build + 1 h prove + 0.5 h wrap = 6 h**. Friday includes repair, not additional scope.

Week 13: Live collection and the instrumented platform

Day	Learn/design — 1.5 h	Build — 3 h	Prove — 1 h
[] D61 Mon	Review source terms, local hardware, Spark topology, and Terraform ownership	Select corridor/categories; acquire tiny real-source samples; pin stack/resource budgets and define network/volume ownership	Confirm source usability, licenses, available resources, and explicit local-only/cloud boundaries.
[] D62 Tue	Study durable landing, checkpoints, and Compose services	Start scoped collectors and scheduled capture; create immutable manifests and Compose startup for reused services	Interrupt and rerun collection; confirm durable raw evidence, safe checkpoints, and secret-free logs.
[] D63 Wed	Study Terraform Docker resources, state, drift, and plan review	Provision reviewed local network/volumes; connect Compose external resources; start Spark master and two workers	Validate/plan before approved apply; submit a job to the workers; demonstrate persistent state and no overlapping ownership.
[] D64 Thu	Study Spark partitioning, deduplication, and atomic publication	Normalize captured records to Parquet; load versioned GTFS/source history through idempotent staging; retain PostGIS matching	Run real inputs and duplicate/changed-record fixtures; verify quarantine, unchanged-version deduplication, and input-to-output lineage.
[] D65 Fri	Study Spark/container metrics, freshness, and alert thresholds	Connect Prometheus scrapes/exporters; build Grafana platform/Spark/data-health dashboards; repair platform blockers	Observe actual metrics and a controlled alert. Friday gate: live collection, a distributed Spark run, laC-owned resources, and working telemetry.

Week 14: Configuration experiments and evidence-backed diagnosis

Day	Learn/design — 1.5 h	Build — 3 h	Prove — 1 h
[] D66 Mon	Study intervals, versioned geography, and replay namespaces	Build minimal dbt/PostGIS overlap/history queries; add pipeline-run/configuration manifests and incident records	Confirm historical cutoff and distance boundaries; replay without duplicated business versions or synthetic city findings.
[] D67 Tue	Study executor memory, partition sizing, skew, and benchmark controls	Establish baseline runs; use AI to propose one Spark configuration patch; benchmark identical captured inputs	Record before/after duration, task/resource evidence, correctness, and rejected proposals. Label amplified replay as load testing.
[] D68 Wed	Study evidence bundles and diagnostic uncertainty	Build read-only AI diagnosis over sanitized metrics, logs, run state, and configuration diffs; add runbook baseline	Trigger isolated memory-pressure and skew scenarios; verify cited observations, uncertainty, and no direct shell/socket access.
[] D69 Thu	Study Compose/Terraform validation and approval boundaries	Add AI patch proposals, saved plan/diff review, and typed wrappers for approved bounded recovery	Reject an unapproved change and an out-of-scope target; validate an approved proposal before execution and retain its audit trail.
[] D70 Fri	Study data failures and held-out incident design	Add schema-change and interrupted-publication scenarios; freeze different held-out variants; repair diagnostic blockers	Compare baseline versus AI on development incidents. Friday gate: four observed incident classes and a usable read-only assistant with approval gates.

Week 15: Recovery proof and the operating case study

Day	Learn/design — 1.5 h	Build — 3 h	Prove — 1 h
[] D71 Mon	Study idempotent retries, bounded backfills, and rollback	Execute an approved retry/backfill or known rollback through controlled wrappers; build the small incident/evidence page	Verify output versions, checkpoint, freshness, and alert recovery. Prove a failed action does not produce a false resolved state.
[] D72 Tue	Review historical knowledge, log hygiene, and operating failure modes	Connect source/run/config/action provenance; exercise interrupted collection, partial feeds, and model outage	Confirm current evidence is distinct from historical knowledge; dashboards/runbooks remain usable without AI and no secret enters model inputs.
[] D73 Wed	Review frozen operational evaluation and timing boundaries	Run held-out incident variants and the runbook baseline; save diagnoses, proposed actions, approved outcomes, and measurements	Report supported diagnosis, abstention, unsafe proposals, recovery time, missing/duplicate versions, and resource overhead without tuning on held-out cases.
[] D74 Thu	Study clean startup, state handling, licensing, and teardown risk	Finish Terraform/Compose instructions, Grafana provisioning, runbooks, attribution, dependency pins, and rollback procedures	Start from an isolated clean environment after approval; replay permitted evidence and verify IaC state/resource ownership without deleting live volumes.
[] D75 Fri	Review the contribution and operational evidence	Repair remaining gates; finish demo, architecture/configuration decisions, incident report, and release package	Demonstrate live data, Spark execution, Grafana alert, cited AI diagnosis, approved recovery, and data correctness. Report unresolved failures and honest empty city findings.

“Release-ready” means the repository is prepared for publication; creating a public repository or publishing source snapshots requires your explicit approval and license review. No external publishing is authorized by this plan.

7. Completion criteria

- [] A bounded city question and permitted 311/CTA/GTFS collectors with seven calendar days of real observations and visible gaps.
- [] A Docker-managed Spark master and two workers run actual normalization/replay jobs; deployment and resource limitations are documented.
- [] Terraform provisions local network/volumes with reviewed plan/state handling and explicit non-overlapping Compose ownership.
- [] Immutable inputs, atomic/idempotent publication, safe checkpoints, source/configuration/run provenance, and versioned geography.
- [] Deterministic overlaps and historical views preserve source uncertainty and exclude later evidence; administrative closure is not physical resolution.
- [] Prometheus scrapes actual metrics; provisioned Grafana dashboards and at least one observed alert show infrastructure, Spark, and data health.
- [] A read-only AI diagnosis path cites bounded operational evidence and offers uncertainty, safe proposals, and a deterministic runbook alternative.
- [] Configuration/recovery proposals pass validation and exact-scope approval; rejected unsafe/unapproved actions and an audit trail have observed results.
- [] Four controlled incident classes plus frozen held-out variants have saved baseline/AI results, failure evidence, and measured recovery checks.
- [] Recovery verifies output correctness, no missing/duplicate business versions, checkpoint state, source freshness, and the alert recovery policy.
- [] Clean-start/replay instructions, Terraform state precautions, model/code licenses, source attribution, runbooks, measured case study, and release package.

A natural city correction/overlap is not guaranteed. Controlled operational failures establish the AIOps experiments, not real city incidents. The completion claim requires observed platform behavior; configuration files and screenshots alone are insufficient. This specification does not implement or deploy the platform.

8. Reuse from the core course

Existing work	Reuse here
R-P5 document processing	Airflow tasks, structured contracts, quarantine, recovery, and review
R-P7 evaluation harness	Frozen incident variants, deterministic baseline, case-level results
R-P8 prompt versioning	Exact diagnosis/proposal prompts and evidence-bundle versions
R-P9 gateway	Local model access, request limits, runtime/resource accounting
R-P10 guardrails	Sanitized operational evidence, secret redaction, typed action boundaries
R-P12 case study	Spark/configuration benchmarks, incident timelines, operating decisions and limits

Your new learning is **Spark operation and tuning, Docker deployment, Terraform state/plan ownership, Prometheus/Grafana instrumentation, AI-assisted incident diagnosis, and approval-gated recovery**, exercised against live-source, replay-safe DE. City-data provenance gives operational claims an inspectable workload rather than a generic chatbot demo.

First action when you start

On D61, verify source terms and tiny samples, inspect available hardware, and define the local stack/resource budget and Terraform/Compose ownership. Start collection on D62. Keep the real city evidence separate from labelled fault/load experiments; both the data product and the operating claims must trace to saved runs.